Self-Calibrating Question Bank Engine Using Real-Time Learner Response Analytics

CSA0925 – Programming in Java

Guided By:
Dr. C. Sivasankar

Presented By:
D. Vahida (192472268)
J. Renuka Devi (192573007)

REVIEW-1 FEEDBACK

Feedback Received

- Ensure careful implementation for multiple students.
- Validate independent student performance.
- Conduct multi-student testing.

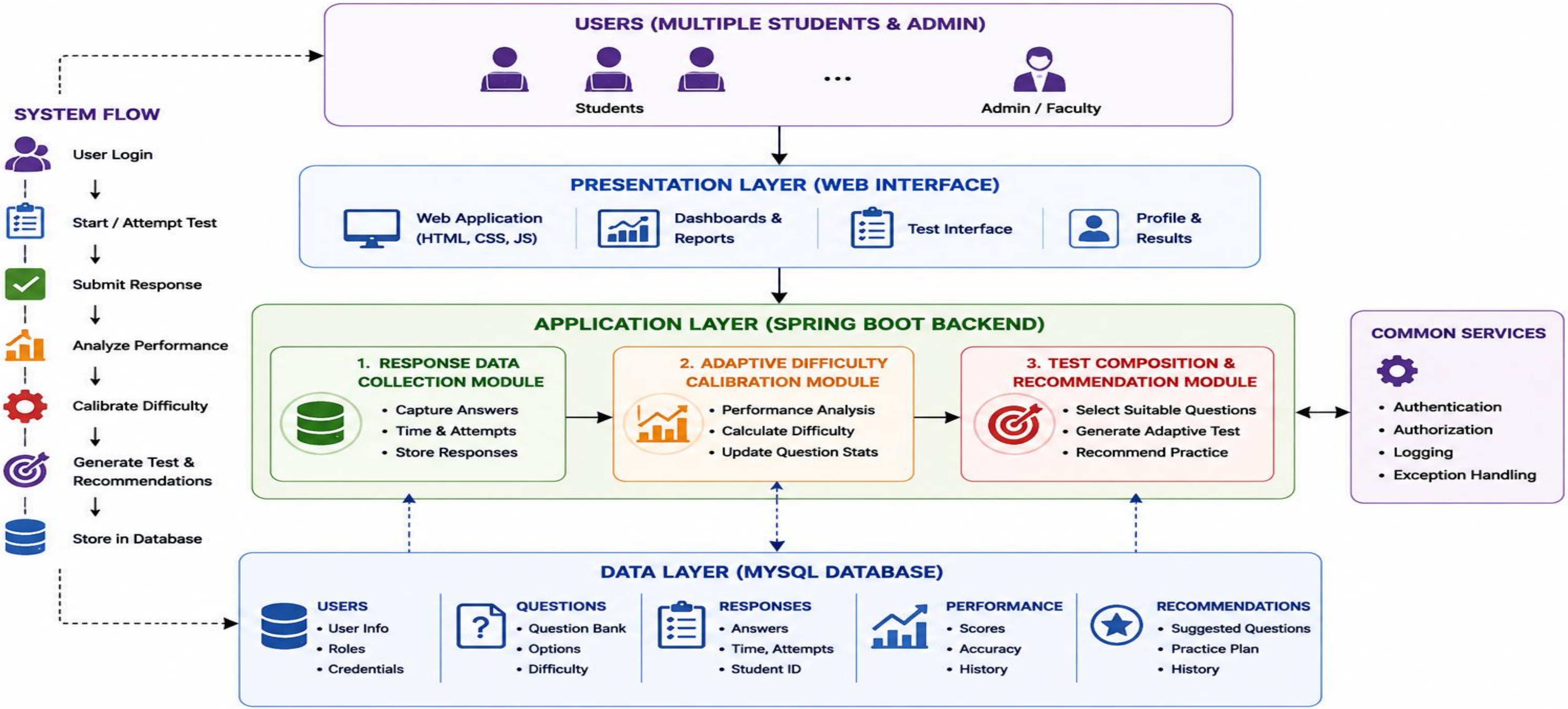
Corrective Actions

- Implemented student-specific response handling.
- Maintained separate performance data for each student.
- Planned multi-student validation with varied performance levels.

Key Focus

Multiple Students → Independent Data → Individual Analysis → Personalized Recommendation

UPDATED ARCHITECTURE



MODULE DESIGN

MODULE 1: RESPONSE DATA COLLECTION

To Collect Real time student data fro the question engine. Includes gathering question details.

MODULE 2 : ADAPTIVE DIFFICULTY CALIBRATION

Dynamically adjust question difficulty based on student performance.

MODULE 3: TEST COMPOSITION & RECOMMENDATION

Create adaptive assessments tailored to individual learning progress.

Data Flow
Student → Response → Database

Interface
Student Response Interface

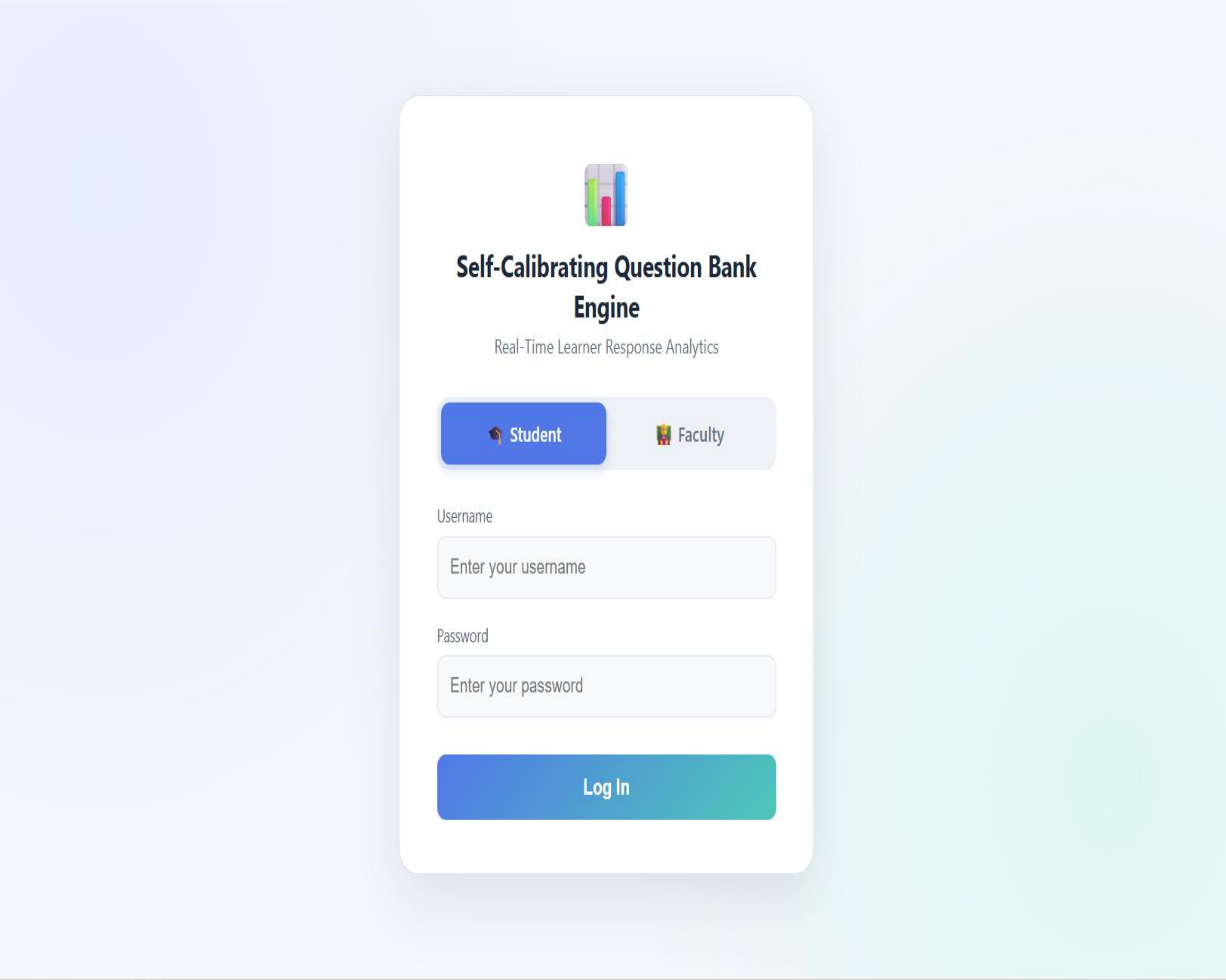
Data Flow
Response → Analysis → Difficulty

Interface
Performance & Difficulty Dashboard

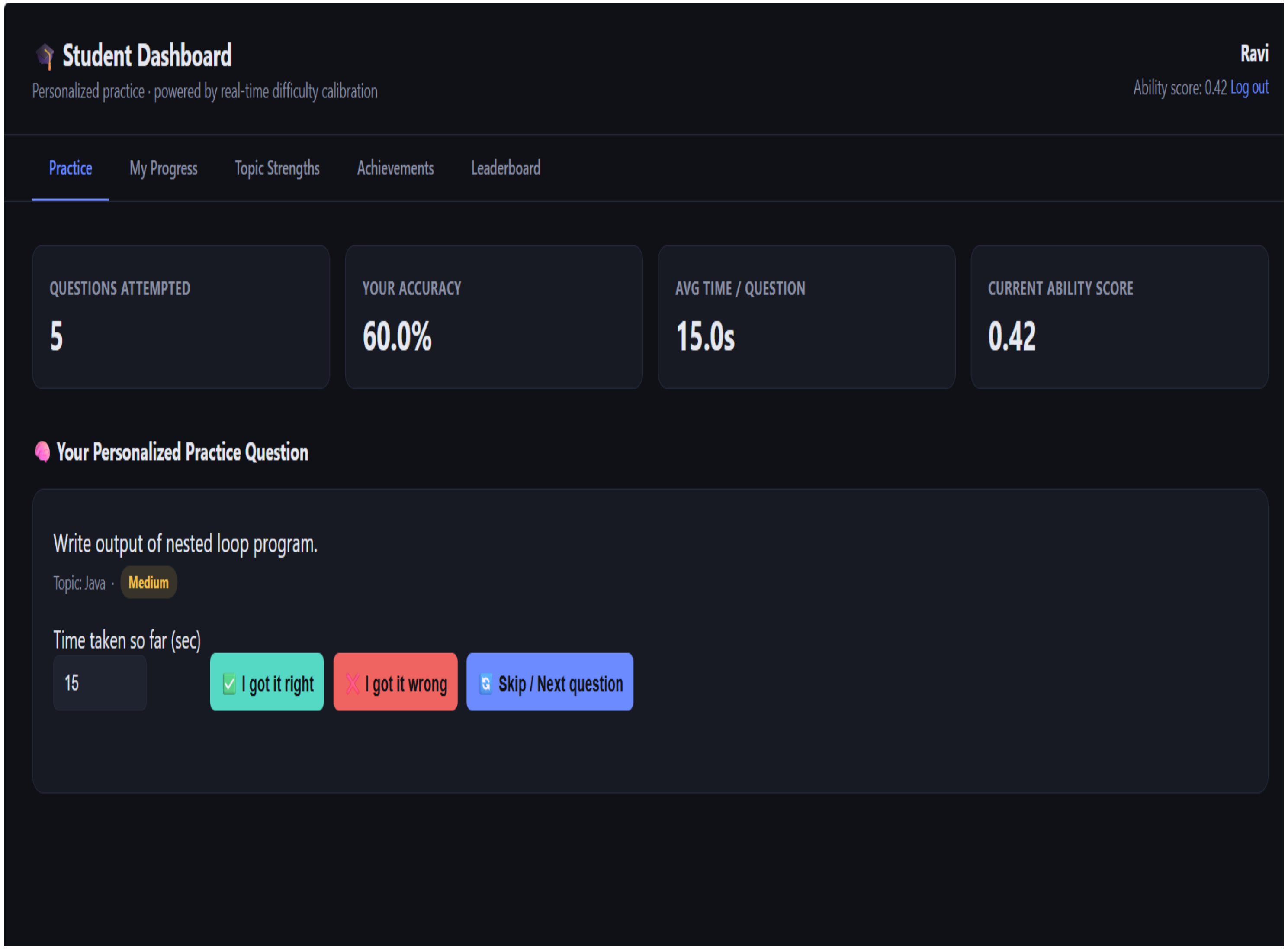
Data Flow
Performance → Selection → Recommendation

Interface
Adaptive Test & Recommendation Interface

IMPLEMENTATION



UI



Dashboard

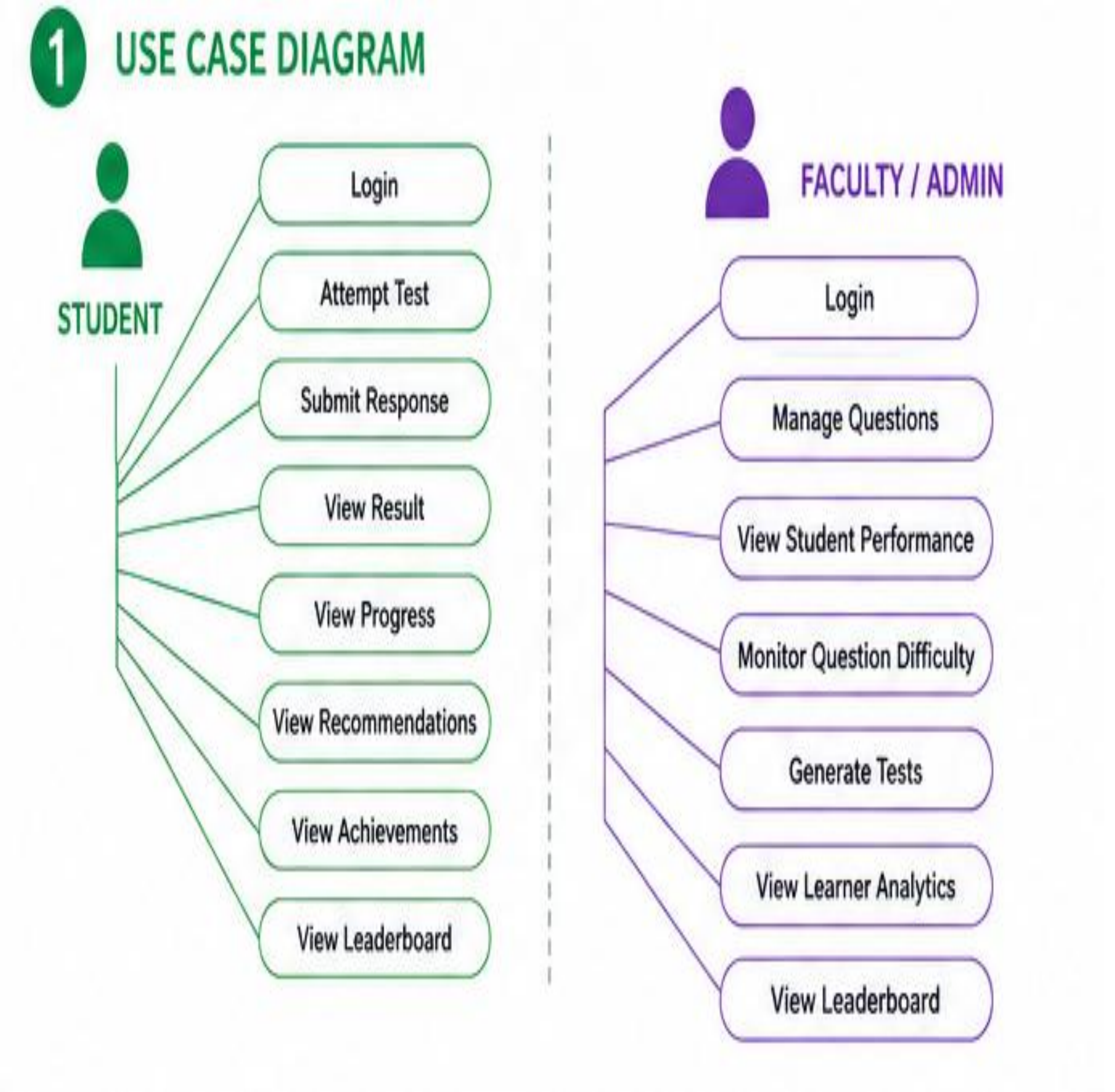
Display:

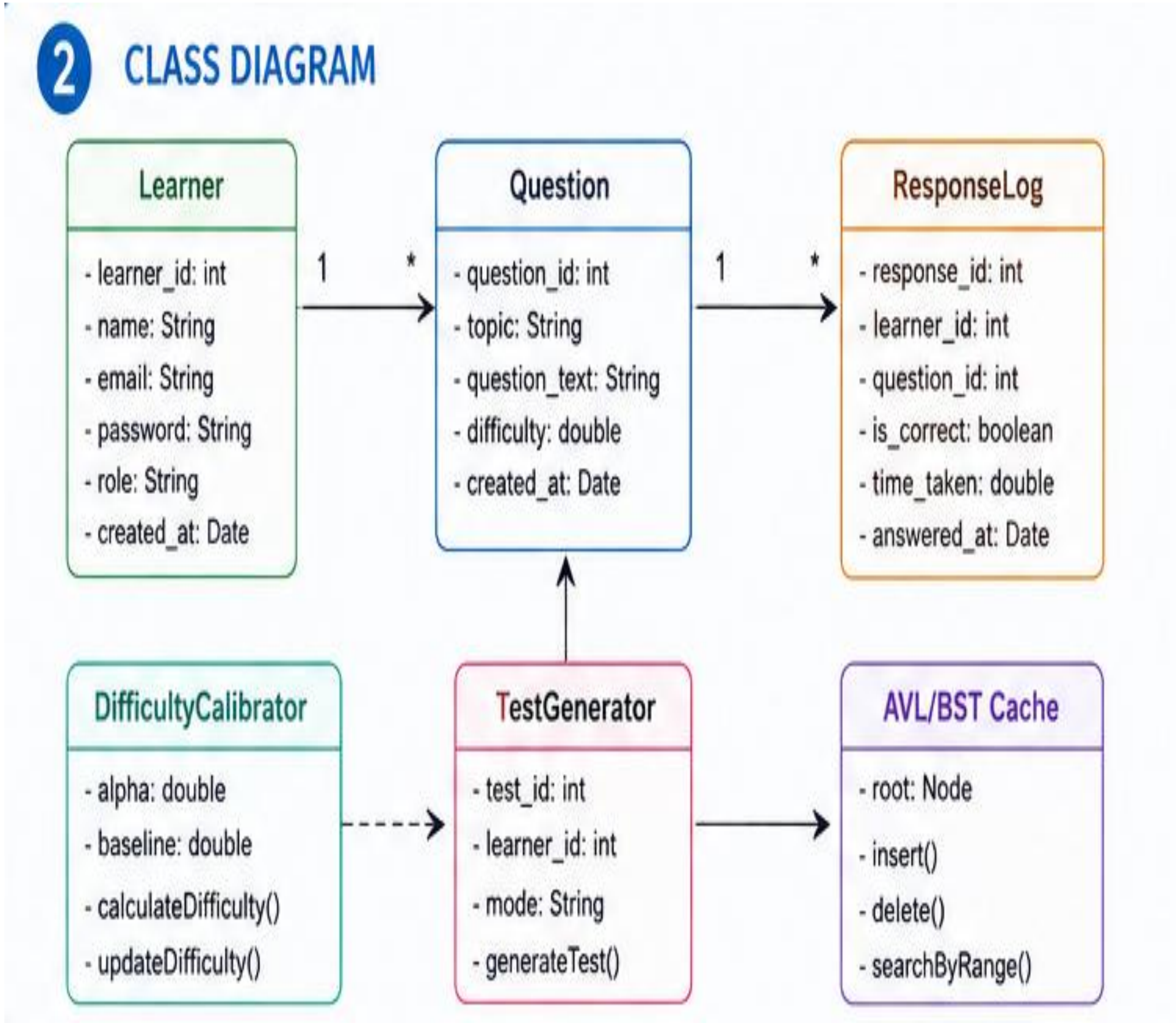
- Total questions attempted
- Correct answers
- Accuracy
- Average response time
- Current difficulty
- Performance trend

Reports

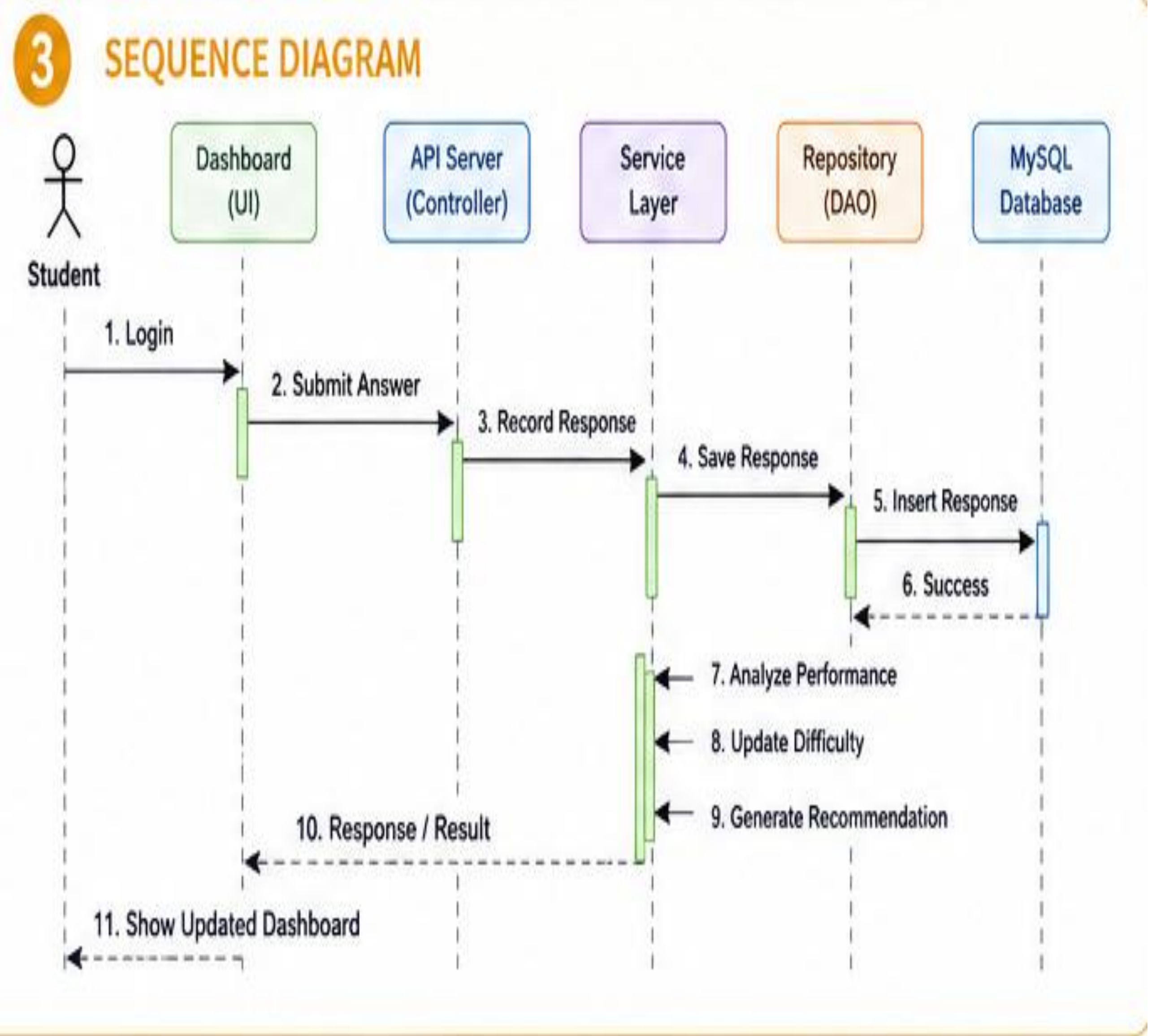
- Learner performance
- Question difficulty
- Weak topics
- Strong topics
- Recommended difficulty

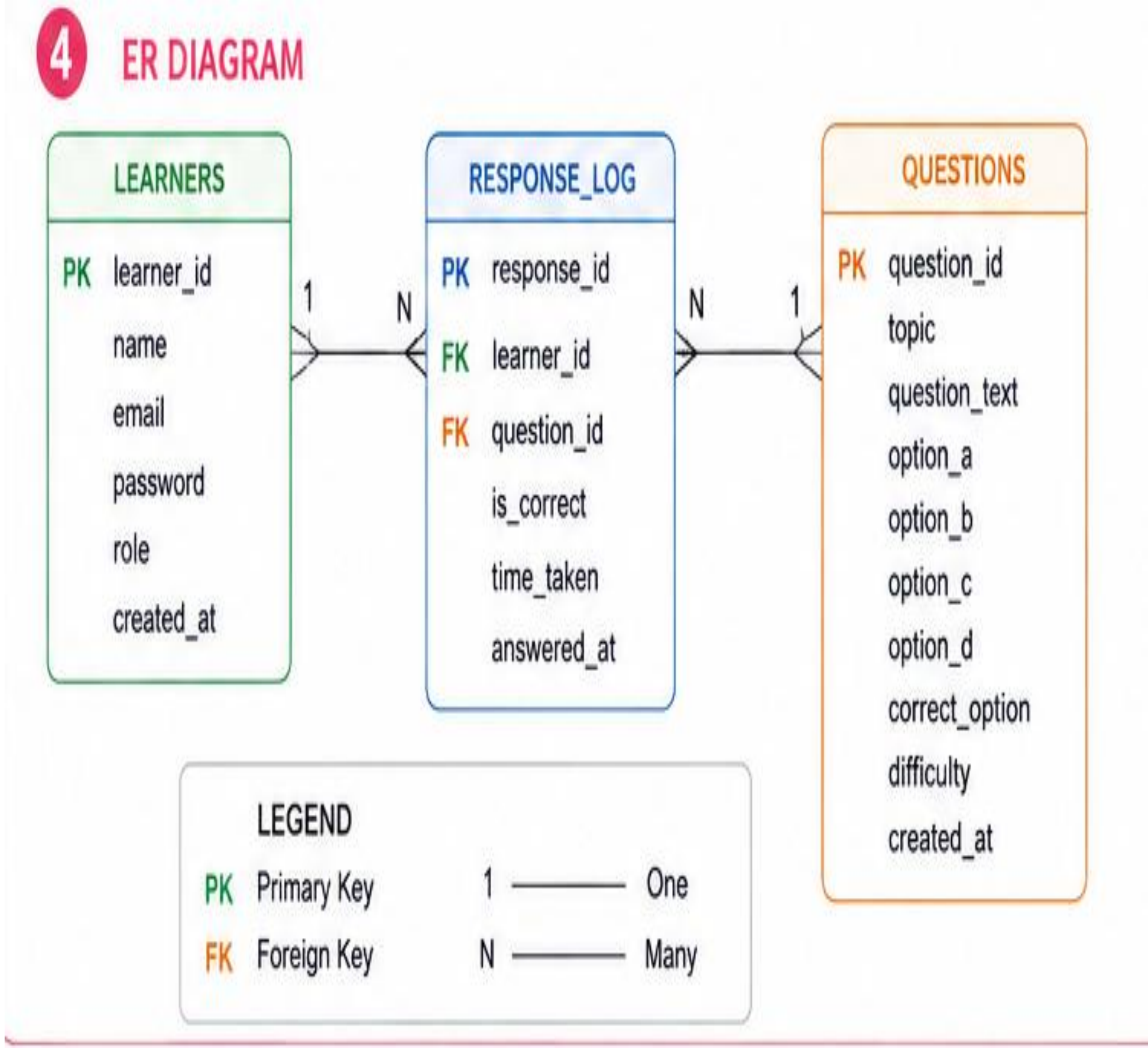
UML DESIGN


1
USE CASE DIAGRAM
STUDENT
Login
Attempt Test
Submit Response
View Result
View Progress
View Recommendations
View Achievements
View Leaderboard
FACULTY / ADMIN
Login
Manage Questions
View Student Performance
Monitor Question Difficulty
Generate Tests
View Learner Analytics
View Leaderboard


2
CLASS DIAGRAM
Learner
- learner_id: int
- name: String
- email: String
- password: String
- role: String
- created_at: Date
1
*
Question
- question_id: int
- topic: String
- question_text: String
- difficulty: double
- created_at: Date
1
*
ResponseLog
- response_id: int
- learner_id: int
- question_id: int
- is_correct: boolean
- time_taken: double
- answered_at: Date
DifficultyCalibrator
- alpha: double
- baseline: double
- calculateDifficulty()
- updateDifficulty()
TestGenerator
- test_id: int
- learner_id: int
- mode: String
- generateTest()
AVL/BST Cache
- root: Node
- insert()
- delete()
- searchByRange()

DATABASE DESIGN





TECHNOLOGY STACK

CATEGORY	TECHNOLOGIES
💻 Languages	Java 21, HTML5, CSS3, JavaScript
⚙️ Frameworks	Java HTTP Server, JDBC
📚 Libraries	MySQL Connector/J
🖥️ Hardware	Windows 11, 8 GB RAM+, Intel Core i5 or equivalent

SYSTEM VERIFICATION MATRIX

Unit Testing Modules

- Login validation
- Question retrieval
- Answer validation
- Difficulty calculation
- Performance calculation

Integration Pipeline Flow

Frontend UI ➔ Spring Boot API ➔ Question Engine ➔ MySQL DB

Test Case	Expected	Result
Valid Login	Dashboard opens	UI Verified
Invalid Login	Error message	UI Verified
Answer Submission	Score increases	UI Verified
Dynamic Difficulty Update	Difficulty adjusts	Scheduled
Adaptive Question Selection	Adaptive question shown	Scheduled

CHALLENGES

Issues:

- Designing dynamic difficulty adjustment.
- Selecting appropriate questions based on learner performance.
- Updating question difficulty without affecting the question bank.

Solutions

- Implemented response analytics.
- Added difficulty-calibration logic.
- Used Spring Boot REST APIs.

Next Plan

Improve calibration accuracy.

Add more learner analytics.

Add topic-wise recommendations.

Thank You